Bayesian statistics with R
8. Heterogeneity and multilevel models (aka mixed models)

Olivier Gimenez

November-December 2025

What is a Generalized Linear Model (or GLM)?

Survival of passengers on the Titanic ~ Class

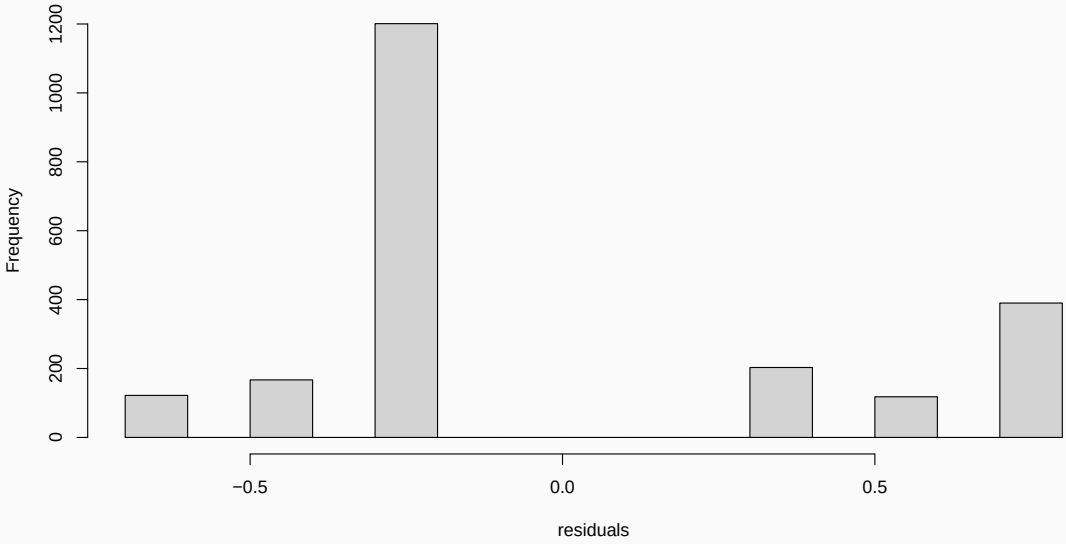
Read `titanic_long.csv` dataset.

```
titanic <- read.csv("dat/titanic_long.csv")
head(titanic)
#>   class   age  sex survived
#> 1 first adult male        1
#> 2 first adult male        1
#> 3 first adult male        1
#> 4 first adult male        1
#> 5 first adult male        1
#> 6 first adult male        1
```

Let's fit a linear model

```
titanic.lm <- lm(survived ~ class, data = titanic)
```

Clearly, the residuals are not normal!



Generalized linear models (GLMs)

- GLMs extend the linear model to scenarios that involve **non-normal error distributions**, hence the term generalized
- The **mean response** is expressed as a **linear function of the predictors** using a **link function**, hence the term linear

Generalized Linear Models

1. **Response variable**
   - Bernoulli/Binomial: Binary variables 0/1
   - Poisson: Counts 0, 1, 2, ...
   - Normal: Real values
   - etc
2. **Predictors** (continuous or categorical)
3. **Link function**
   - Gaussian: identity
   - Binomial: logit
   - Poisson: log
   - Type in `?family`

Bernoulli/Binomial distribution (logistic regression)

- Response variable: Yes/No (e.g. dead/alive, male/female, presence/absence)
- Link function: `logit`

$$\text{logit}(p) = \ln\left(\frac{p}{1-p}\right)$$

- Then, if predictor is x

$$\begin{aligned}
\text{Response} &\sim \text{Distribution}(\text{Mean Response}) \\
Y_i &\sim \text{Bernoulli}(p_i) \\
\text{logit}(p_i) &= a + b\, x_i \\
p_i = \text{logit}^{-1}(a + b\, x_i) &= \frac{e^{a+b\, x_i}}{1 + e^{a+b\, x_i}}
\end{aligned}$$

Poisson distributoin (Poisson regression)

- Response variable: Counts (0, 1, 2, 3...)
- Link function: `log`

$$\text{Response} \sim \text{Distribution}(\text{Mean Response})$$
$$Y_i \sim \text{Poisson}(\lambda_i)$$
$$\log(\lambda_i) = a + b\, x_i$$
$$\lambda_i = e^{a+b\, x_i}$$

Back to survival of Titanic passengers

How many passengers traveled in each class?

```
tapply(titanic$survived, titanic$class, length)
#>   crew  first second  third
#>    885    325    285    706
```

Back to survival of Titanic passengers

How many survived?

```
tapply(titanic$survived, titanic$class, sum)
#>   crew  first second  third
#>    212    203    118    178
```

Back to survival of Titanic passengers

What proportion survived in each class?

```
as.numeric(tapply(titanic$survived, titanic$class, mean))
#> [1] 0.2395480 0.6246154 0.4140351 0.2521246
```

Back to survival of Titanic passengers (package `dplyr`)

Arrange passenger survival according to class

```
library(dplyr)
summarise(group_by(titanic, class, survived), count = n())
```

Back to survival of Titanic passengers (package `dplyr`)

Same manipulation using the pipe operator %>%

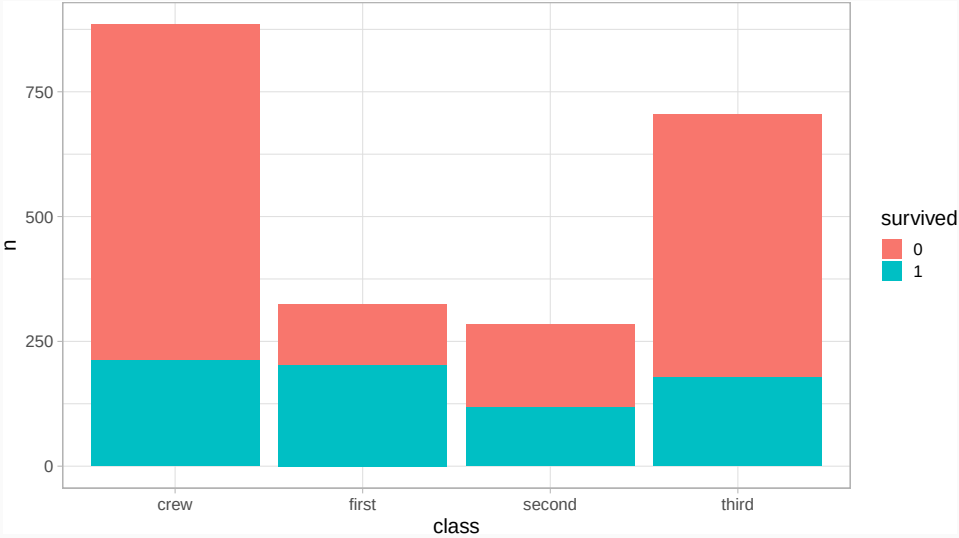
```
titanic %>%
  group_by(class, survived) %>%
  summarise(count = n())
```

Back to survival of Titanic passengers (package `dplyr`)

Arrange passenger survival according to class

```
#>    class survived   n
#> 1   crew        0 673
#> 2  third        0 528
#> 3   crew        1 212
#> 4  first        1 203
#> 5  third        1 178
#> 6 second        0 167
#> 7  first        0 122
#> 8 second        1 118
```

Or graphically. . .



Fitting GLMs in R: `glm` function

```
titanic.glm <- glm(survived ~ class, data = titanic, family = binomial)
#> # A tibble: 4 x 5
#>   term        estimate std.error statistic  p.value
#>   <chr>          <dbl>     <dbl>     <dbl>    <dbl>
#> 1 (Intercept)  -1.16      0.0788   -14.7   1.05e-48
#> 2 classfirst    1.66      0.139     12.0   4.97e-33
#> 3 classsecond   0.808     0.144      5.62  1.91e- 8
#> 4 classthird    0.0678    0.117      0.579 5.62e- 1
```

These estimates are on the logit scale!

Interpreting logistic regression outputs

Parameter estimates on the logit scale:

```
#> (Intercept)  classfirst classsecond  classthird
#> -1.15515905  1.66434399  0.80784987  0.06784632
```

We need to back-transform using the inverse logit function:

```
plogis(coef(titanic.glm)[1]) # crew survival probability
#> (Intercept)
#>    0.239548
```

Looking at the data, the proportion of crew who survived is:

```
sum(titanic$survived[titanic$class == "crew"]) /
  nrow(titanic[titanic$class == "crew", ])
#> [1] 0.239548
```

Probability of survival for 1st class passengers?

Needs to add intercept (baseline) to the parameter estimate:

```
plogis(coef(titanic.glm)[1] + coef(titanic.glm)[2])
#> (Intercept)
#>   0.6246154
```

Again this value matches the data:

```
sum(titanic$survived[titanic$class == "first"]) /
  nrow(titanic[titanic$class == "first", ])
#> [1] 0.6246154
```

Multilevel (aka mixed-effect) models

What are multilevel models?

- Multilevel models include both fixed and random effects.

What are multilevel models?

- Multilevel models include both fixed and random effects.
- Random effects are statistical parameters that attempt to **explain noise caused by clusters** of the population you are trying to model.

What are multilevel models?

- Multilevel models include both fixed and random effects.
- Random effects are statistical parameters that attempt to **explain noise caused by clusters** of the population you are trying to model.
- A multilevel model assumes that the dataset being analysed consists of **a hierarchy of different populations** whose differences relate to that hierarchy.

What are multilevel models?

- Multilevel models include both fixed and random effects.
- Random effects are statistical parameters that attempt to **explain noise caused by clusters** of the population you are trying to model.
- A multilevel model assumes that the dataset being analysed consists of **a hierarchy of different populations** whose differences relate to that hierarchy.
- Measurement that come **in clusters** or groups.

What are multilevel models?

- Multilevel models include both fixed and random effects.
- Random effects are statistical parameters that attempt to **explain noise caused by clusters** of the population you are trying to model.
- A multilevel model assumes that the dataset being analysed consists of **a hierarchy of different populations** whose differences relate to that hierarchy.
- Measurement that come **in clusters** or groups.
- Come up with examples of clusters or groups.

Clusters might be:

- Classrooms within schools
- Students within classrooms
- Chapters within books
- Individuals within populations
- Populations within species
- Trajectories within individuals
- Fishes within tanks
- Frogs within ponds
- PhD applicants in doctoral schools
- Nations in continents
- Sex or age are not clusters per se (if we were to sample again, we would take the same levels, e.g. male/female and young/old)

Why do we need multilevel models?

- Model the clustering itself.

Why do we need multilevel models?

- Model the clustering itself.
- Interested in variance components (environmental vs. genetic variance).

Why do we need multilevel models?

- Model the clustering itself.
- Interested in variance components (environmental vs. genetic variance).
- Control for bias due to pseudoreplication (time, space, individual).

McElreath's explanation of multilevel models

- Fixed-effect models have amnesia.

McElreath's explanation of multilevel models

- Fixed-effect models have amnesia.
- Every new cluster (individual, species, classroom) is a new world.

McElreath's explanation of multilevel models

- Fixed-effect models have amnesia.
- Every new cluster (individual, species, classroom) is a new world.
- No information passed among clusters.

McElreath's explanation of multilevel models

- Fixed-effect models have amnesia.
- Every new cluster (individual, species, classroom) is a new world.
- No information passed among clusters.
- Multilevel models remember and pool information. They have memory.

McElreath's explanation of multilevel models

- Fixed-effect models have amnesia.
- Every new cluster (individual, species, classroom) is a new world.
- No information passed among clusters.
- Multilevel models remember and pool information. They have memory.
- Properties of clusters come from a population.

McElreath's explanation of multilevel models

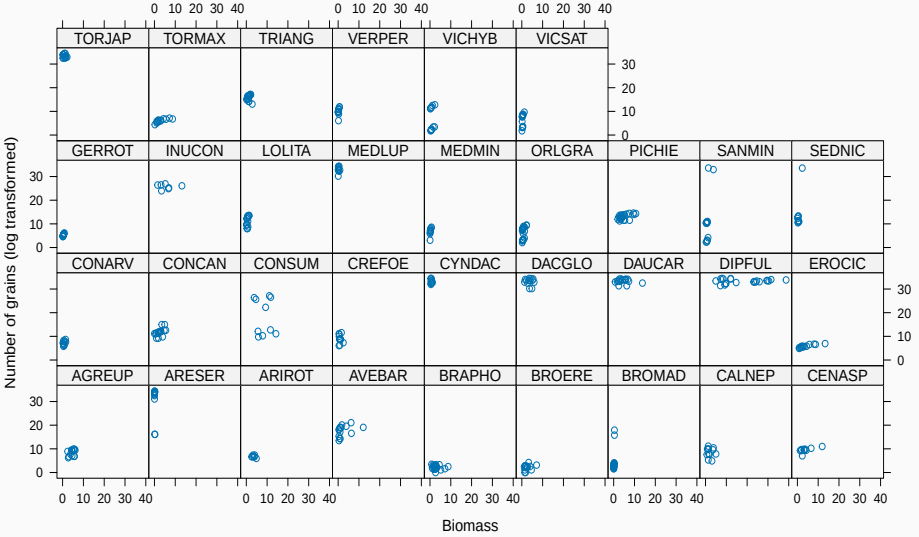
- Fixed-effect models have amnesia.
- Every new cluster (individual, species, classroom) is a new world.
- No information passed among clusters.
- Multilevel models remember and pool information. They have memory.
- Properties of clusters come from a population.
- If previous clusters improve your guess about a new cluster, you want to use pooling.

Plant experiment in the field at CEFE



Courtesy of Pr Eleni Kazakou

Number of grains per species (cluster) as a function of biomass



GLM with complete pooling

$$
\begin{aligned}
\mathrm{Y}_i &\sim \text{Distribution}(\text{mean}_i) && \text{[likelihood]} \\
\text{link}(\text{mean})_i &= \alpha + \beta\, x_i && \text{[linear model]} \\
\alpha &\sim \text{to be determined} && \text{[prior for intercept]} \\
\beta &\sim \text{to be determined} && \text{[prior for slope]}
\end{aligned}
$$

Model with complete pooling. All clusters the same.

GLM with no pooling

$$
\begin{aligned}
\mathrm{Y}_i &\sim \text{Distribution}(\text{mean}_i) && \text{[likelihood]} \\
\text{link}(\text{mean})_i &= \alpha_{\text{CLUSTER}[i]} + \beta\, x_i && \text{[linear model]} \\
\alpha_j &\sim \text{to be determined} && \text{[prior for intercept]} \\
\beta &\sim \text{to be determined} && \text{[prior for slope]}
\end{aligned}
$$

Model with no pooling. All clusters unrelated (fixed effect).

GLMM or GLM with partial pooling

$$
\begin{aligned}
\mathrm{Y}_i &\sim \text{Distribution}(\text{mean}_i) && \text{[likelihood]} \\
\text{link}(\text{mean})_i &= \alpha_{\text{CLUSTER}[i]} + \beta\, x_i && \text{[linear model]} \\
\alpha_j &\sim \text{Normal}(\bar{\alpha}, \sigma) && \text{[prior for varying intercepts]} \\
\bar{\alpha} &\sim \text{to be determined} && \text{[prior for population mean]} \\
\sigma &\sim \text{to be determined} && \text{[prior for standard deviation]} \\
\beta &\sim \text{to be determined} && \text{[prior for slope]}
\end{aligned}
$$

Model with partial pooling. Clusters are somehow related (random effect).

Back to the plant example

Model with complete pooling (all species are the same)

$$\text{nseeds}_i \sim \text{Normal}(\mu_i, \sigma^2) \quad \text{[likelihood]}$$

$$\mu_i = \alpha + \beta\ \text{biomass}_i \quad \text{[linear model]}$$

$$\alpha \sim \text{Normal}(0, 1000) \quad \text{[prior for intercept]}$$

$$\beta \sim \text{Normal}(0, 1000) \quad \text{[prior for slope]}$$

$$\sigma \sim \text{Uniform}(0, 100) \quad \text{[prior for standard deviation]}$$

Read in and manipulate data

```r
# read in data
VMG <- read_csv2(here::here("slides","dat","VMG.csv")) %>%
  mutate(Sp = as_factor(Sp), Vm = as.numeric(Vm))
# nb of seeds
y <- log(VMG$NGrTotest)
# biomass
x <- VMG$Vm
x <- (x - mean(x))/sd(x)
# species name
Sp <- VMG$Sp
# species label
species <- as.numeric(Sp)
# species name
nbspecies <- length(levels(Sp))
# total nb of measurements
n <- length(y)
```

Specify the model in Jags

```
model <-
paste("
model{
for(i in 1:n){
    y[i] ~ dnorm(mu[i], tau.y)
    mu[i] <- a + b * x[i]
    }
tau.y <- 1 / (sigma.y * sigma.y)
sigma.y ~ dunif(0,100)
a ~ dnorm(0,0.001)
b ~ dnorm(0,0.001)
}
")
writeLines(model,here::here("slides","code","completepooling.bug"))
```

Prepare ingredients for running Jags

```
# data
allom.data <- list(y = y, n = n, x = x)

# initial values
init1 <- list(a=rnorm(1), b=rnorm(1),sigma.y=runif(1))
init2 <- list(a=rnorm(1), b=rnorm(1),sigma.y=runif(1))
inits <- list(init1,init2)

# parameters to be estimated
allom.parameters <- c("a", "b", "sigma.y")
```

Run Jags

```
allom.1 <- jags(allom.data,
                inits,
                allom.parameters,
                n.iter = 2500,
                model.file = here::here("slides","code","completepooling.bug"),
                n.chains = 2,
                n.burn = 1000)
#> Compiling model graph
#>    Resolving undeclared variables
#>    Allocating nodes
#> Graph information:
#>    Observed stochastic nodes: 488
#>    Unobserved stochastic nodes: 3
#>    Total graph size: 1956
#>
#> Initializing model
```

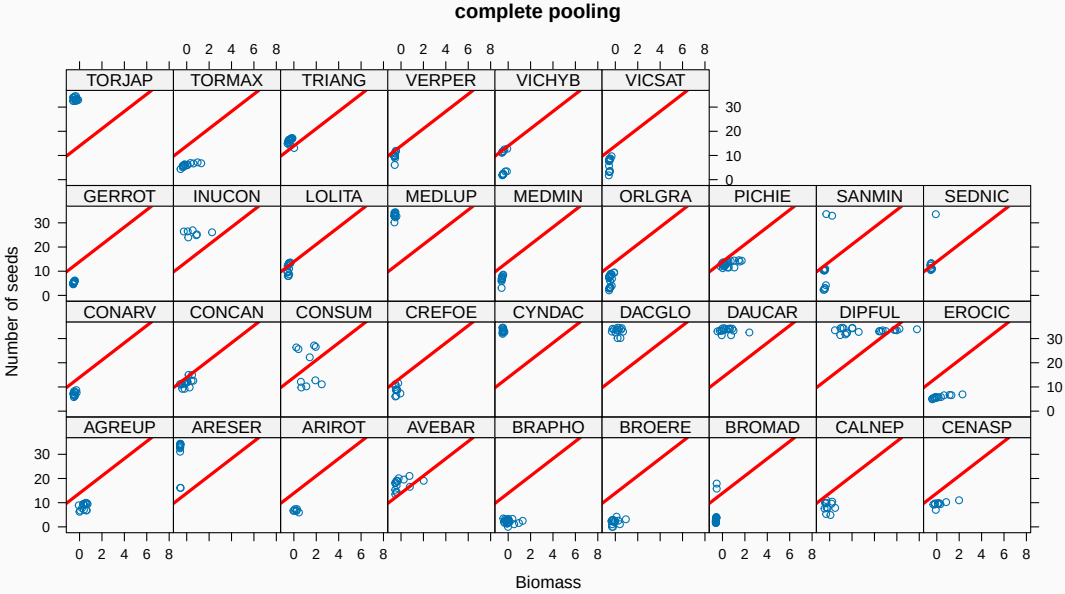
Display results

```
allom.1
#> Inference for Bugs model at "/Users/oliviergimenez/Dropbox/OG/GITHUB/bayesian-stats-wit
#>  2 chains, each with 2500 iterations (first 1000 discarded)
#>  n.sims = 3000 iterations saved. Running time = 0.489 secs
#>           mu.vect sd.vect     2.5%      25%      50%      75%    97.5%  Rhat
#> a          13.919   0.472   13.003   13.592   13.922   14.236   14.811 1.001
#> b           3.569   0.471    2.636    3.255    3.576    3.894    4.474 1.001
#> sigma.y    10.422   0.348    9.778   10.187   10.412   10.650   11.134 1.003
#> deviance 3672.091   2.459 3669.225 3670.315 3671.454 3673.215 3678.320 1.003
#>          n.eff
#> a         3000
#> b         2200
#> sigma.y   3000
#> deviance   610
#>
#> For each parameter, n.eff is a crude measure of effective sample size,
#> and Rhat is the potential scale reduction factor (at convergence, Rhat=1).
```

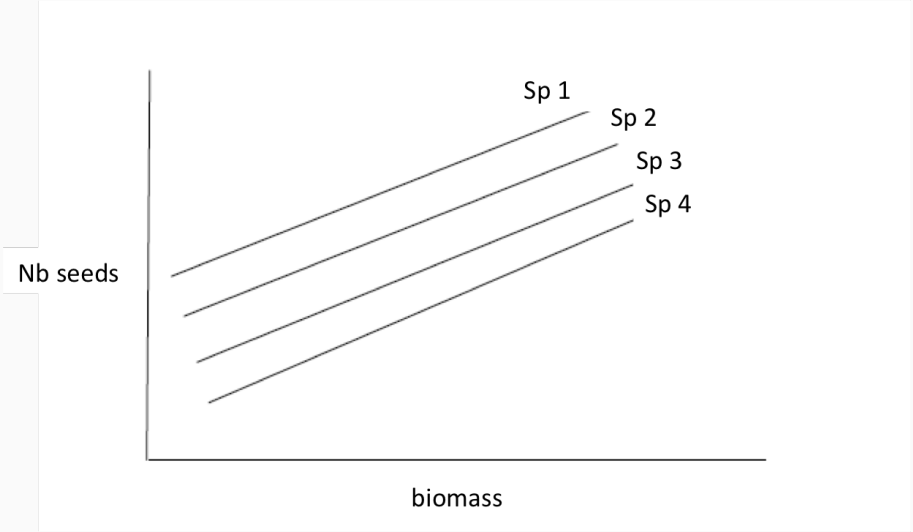
Compare with Frequentist approach

```
freq_lm <- lm(y ~ x, data = allom.data)
freq_lm
#>
#> Call:
#> lm(formula = y ~ x, data = allom.data)
#>
#> Coefficients:
#> (Intercept)            x
#>      13.927        3.578
```

Output


complete pooling
TORJAP
TORMAX
TRIANG
VERPER
VICHYB
VICSAT
GERROT
INUCON
LOLITA
MEDLUP
MEDMIN
ORLGRA
PICHIE
SANMIN
SEDNIC
CONARV
CONCAN
CONSUM
CREFOE
CYNDAC
DACGLO
DAUCAR
DIPFUL
EROCIC
AGREUP
ARESER
ARIROT
AVEBAR
BRAPHO
BROERE
BROMAD
CALNEP
CENASP
Number of seeds
Biomass

Model with partial pooling (species random effect)



Model with partial pooling (all species related in some way)

$$\text{nseeds}_i \sim \text{Normal}(\mu_i, \sigma^2) \quad \text{[likelihood]}$$

$$\mu_i = \alpha_{\text{species}[i]} + \beta\ \text{biomass}_i \quad \text{[linear model]}$$

$$\alpha_j \sim \text{Normal}(\bar{\alpha}, \sigma_\alpha) \quad \text{[prior for varying intercepts]}$$

$$\bar{\alpha} \sim \text{Normal}(0, 1000) \quad \text{[prior for population mean]}$$

$$\sigma_\alpha \sim \text{Uniform}(0, 100) \quad \text{[prior for } \sigma_\alpha\text{]}$$

$$\beta \sim \text{Normal}(0, 1000) \quad \text{[prior for slope]}$$

$$\sigma \sim \text{Uniform}(0, 100) \quad \text{[prior for } \sigma\text{]}$$

Implementation in Jags

```
model <- paste("
model {
  for (i in 1:n){
    y[i] ~ dnorm(mu[i], tau.y)
    mu[i] <- a[species[i]] + b * x[i]
  }
  tau.y <- 1/ (sigma.y * sigma.y)
  sigma.y ~ dunif(0, 100)
  for (j in 1:nbspecies){
    a[j] ~ dnorm(mu.a, tau.a)
  }
  mu.a ~ dnorm(0, 0.001)
  tau.a <- 1/(sigma.a * sigma.a)
  sigma.a ~ dunif(0, 100)
  b ~ dnorm (0, 0.001)
}")
writeLines(model,here::here("slides","code","varint.bug"))
```

Prepare ingredients for running Jags

```
allom.data <- list(n = n,
                   nbspecies = nbspecies,
                   x = x,
                   y = y,
                   species = species)
init1 <- list(a = rnorm(nbspecies), b = rnorm(1), mu.a = rnorm(1),
              sigma.y = runif(1), sigma.a=runif(1))
init2 <- list(a = rnorm(nbspecies), b = rnorm(1), mu.a = rnorm(1),
              sigma.y = runif(1), sigma.a = runif(1))
inits <- list(init1,init2)
allom.parameters <- c("b", "mu.a","sigma.y", "sigma.a")
```

Run Jags

```
allom.2 <- jags(allom.data,
                inits,
                allom.parameters,
                n.iter = 2500,
                model.file = here::here("slides","code","varint.bug"),
                n.chains = 2,
                n.burn = 1000)
#> Compiling model graph
#>    Resolving undeclared variables
#>    Allocating nodes
#> Graph information:
#>    Observed stochastic nodes: 488
#>    Unobserved stochastic nodes: 37
#>    Total graph size: 2484
#>
#> Initializing model
```

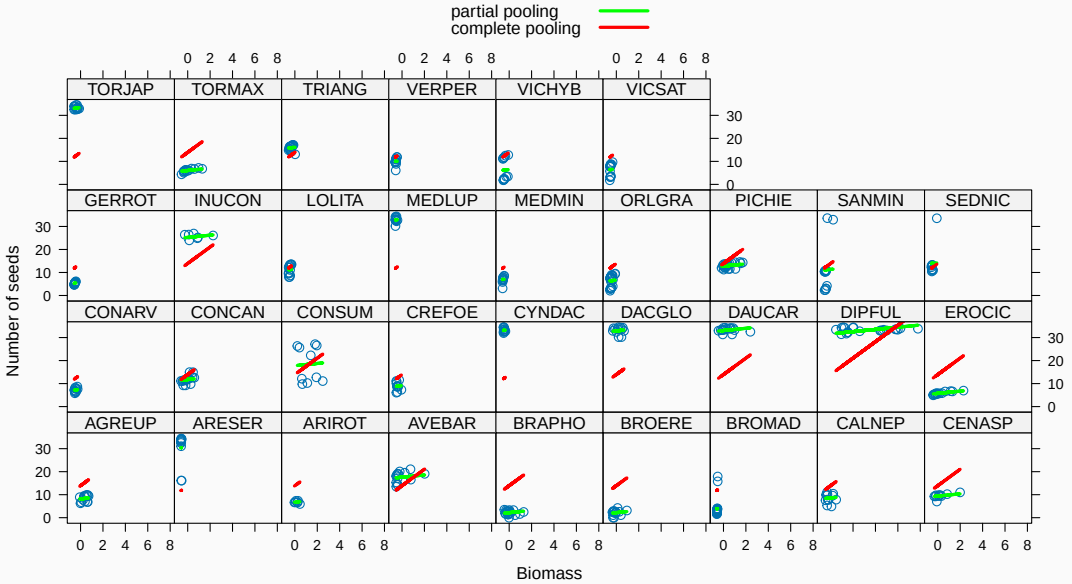
Display results

```
allom.2
#> Inference for Bugs model at "/Users/oliviergimenez/Dropbox/OG/GITHUB/bayesian-stats-wit
#>  2 chains, each with 2500 iterations (first 1000 discarded)
#>  n.sims = 3000 iterations saved. Running time = 0.525 secs
#>           mu.vect sd.vect     2.5%      25%      50%      75%    97.5%  Rhat
#> b           0.473   0.242   -0.004    0.314    0.476    0.639    0.949 1.002
#> mu.a       14.424   1.926   10.657   13.157   14.378   15.734   18.259 1.003
#> sigma.a    11.109   1.473    8.661   10.074   10.958   11.979   14.371 1.006
#> sigma.y     3.070   0.101    2.880    3.000    3.067    3.137    3.279 1.002
#> deviance 2478.141   8.788 2462.800 2471.737 2477.567 2483.499 2497.602 1.001
#>          n.eff
#> b         1400
#> mu.a      1400
#> sigma.a    460
#> sigma.y   1100
#> deviance  3000
#>
```

Compare with Frequentist approach

```
library(lme4)
freq_lmm <- lmer(y ~ x + (1 | species), allom.data, REML = FALSE)
freq_lmm
#> Linear mixed model fit by maximum likelihood  ['lmerMod']
#> Formula: y ~ x + (1 | species)
#>    Data: allom.data
#>       AIC       BIC    logLik -2*log(L)  df.resid
#>  2652.606  2669.368 -1322.303  2644.606       484
#> Random effects:
#>  Groups   Name        Std.Dev.
#>  species  (Intercept) 10.472
#>  Residual              3.058
#> Number of obs: 488, groups:  species, 33
#> Fixed Effects:
#> (Intercept)            x
#>      14.526        0.479
```

Compare complete pooling vs partial pooling



Model with no pooling (all species unrelated)

$$
\begin{aligned}
\text{nseeds}_i &\sim \text{Normal}(\mu_i, \sigma^2) && \text{[likelihood]} \\
\mu_i &= \alpha_{\text{species}[i]} + \beta\ \text{biomass}_i && \text{[linear model]} \\
\alpha_j &\sim \text{Normal}(0, 1000) && \text{[prior for intercepts]} \\
\beta &\sim \text{Normal}(0, 1000) && \text{[prior for slope]} \\
\sigma &\sim \text{Uniform}(0, 100) && \text{[prior for}\sigma\text{]}
\end{aligned}
$$

Implementation in Jags

```
model <- paste("
model {
  for (i in 1:n){
    y[i] ~ dnorm (mu[i], tau.y)
    mu[i] <- a[species[i]] + b * x[i]
  }
  tau.y <- 1 / (sigma.y * sigma.y)
  sigma.y ~ dunif(0, 100)
  for (j in 1:nbspecies){
    a[j] ~ dnorm(0, 0.001)
  }
  b ~ dnorm(0,0.1)
}")
writeLines(model,here::here("slides","code","nopooling.bug"))
```

Prepare ingredients

```
allom.data <- list(n = n,
                   nbspecies = nbspecies,
                   x = x,
                   y = y,
                   species = species)
init1 <- list(a = rnorm(nbspecies), b = rnorm(1), sigma.y = runif(1))
init2 <- list(a = rnorm(nbspecies), b = rnorm(1), sigma.y = runif(1))
inits<-list(init1, init2)
allom.parameters <- c("a","b","sigma.y")
```

Run JAGS

```
allom.3 <- jags(data = allom.data,
                inits = inits,
                parameters.to.save = allom.parameters,
                n.iter = 2500,
                model.file = here::here("slides","code","nopooling.bug"),
                n.chains = 2,
                n.burn = 1000)
#> Compiling model graph
#>    Resolving undeclared variables
#>    Allocating nodes
#> Graph information:
#>    Observed stochastic nodes: 488
#>    Unobserved stochastic nodes: 35
#>    Total graph size: 2481
#>
#> Initializing model
```

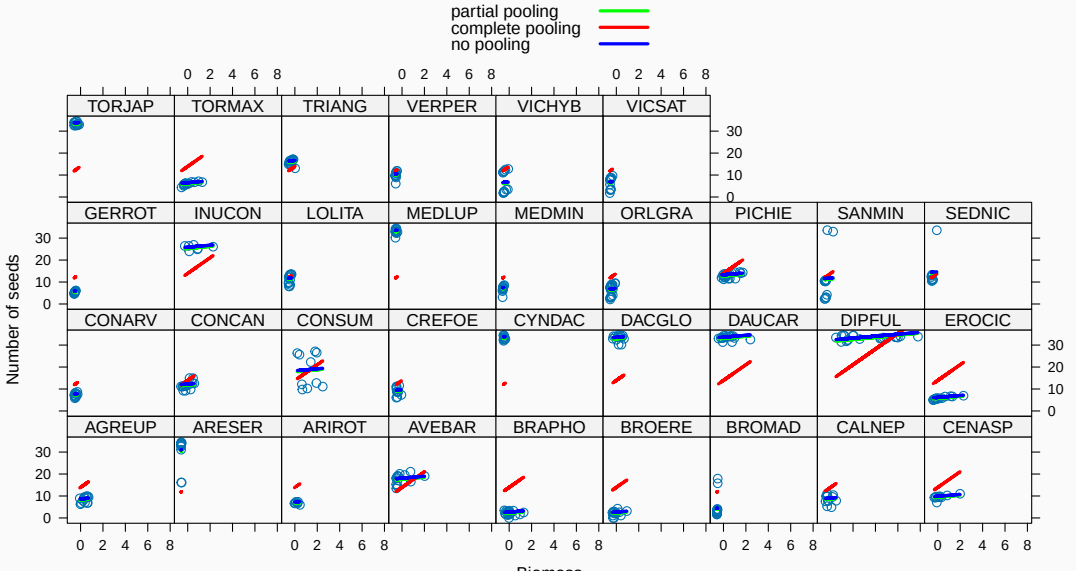
Display results

```
allom.3$BUGSoutput$summary[c(1:4, 32:33, 34), -c(4,6)]
#>              mean        sd        2.5%       50%      97.5%     Rhat n.eff
#> a[1]    8.1588743 0.8441862  6.44500516  8.158973  9.7769529 1.001144  3000
#> a[2]   30.7821745 0.8844459 29.06940511 30.790184 32.4805309 1.000813  3000
#> a[3]    6.7015552 1.1668311  4.43745335  6.729674  8.9484538 1.000863  3000
#> a[4]   17.6113289 0.7990702 16.00322281 17.585908 19.1830738 1.002720   730
#> a[32]   6.3647381 0.8030463  4.79693819  6.383268  7.9275451 1.001212  2700
#> a[33]   6.6155501 0.8045362  5.02736929  6.636236  8.1615371 1.004054   990
#> b       0.4401534 0.2456820 -0.03406992  0.437875  0.9192433 1.000695  3000
```

Compare with Frequentist approach

```
lm(y ~ -1 + as.factor(species) + x, data = allom.data) %>%
  broom::tidy() %>%
  slice(c(1:4, 32:33, 34))
#> # A tibble: 7 x 5
#>   term                  estimate std.error statistic   p.value
#>   <chr>                    <dbl>     <dbl>     <dbl>     <dbl>
#> 1 as.factor(species)1      8.17      0.824      9.92 3.92e- 21
#> 2 as.factor(species)2     30.8       0.895     34.4  1.67e-128
#> 3 as.factor(species)3      6.67      1.16       5.76 1.56e-  8
#> 4 as.factor(species)4     17.6       0.791     22.3  5.32e- 75
#> 5 as.factor(species)32     6.38      0.797      8.01 9.95e- 15
#> 6 as.factor(species)33     6.63      0.800      8.29 1.33e- 15
#> 7 x                        0.441     0.243      1.81 7.06e-  2
```

Compare complete pooling vs partial pooling vs no pooling



Bonus: Model with varying intercept and varying slope

Code: part 1

```
model <-
paste("
# varying-intercept, varying-slope allometry model
# with Vm as a species predictor

model {
  for (i in 1:n){
    y[i] ~ dnorm (mu[i], tau.y)
    mu[i] <- a[species[i]] + b[species[i]] * x[i]
  }

  tau.y <- pow(sigma.y, -2)
  sigma.y ~ dunif (0, 100)
```

Code: part 2

```
  for (j in 1:nbspecies){
    a[j] ~ dnorm (mu.a, tau.a)
    b[j] ~ dnorm (mu.b, tau.b)
  }

  mu.a ~ dnorm (0, .001)
  tau.a <- pow(sigma.a, -2)
  sigma.a ~ dunif (0, 100)
  mu.b ~ dnorm (0, .001)
  tau.b <- pow(sigma.b, -2)
  sigma.b ~ dunif (0, 100)

}
")
```

Prepare ingredients

```
init1 <- list(a = rnorm(nbspecies), b = rnorm(nbspecies),
              mu.a = rnorm(1), mu.b = rnorm(1),
              sigma.y = runif(1), sigma.a = runif(1), sigma.b = runif(1))
init2 <- list(a = rnorm(nbspecies), b = rnorm(nbspecies),
              mu.a = rnorm(1), mu.b = rnorm(1),
              sigma.y = runif(1), sigma.a = runif(1), sigma.b = runif(1))
inits <- list(init1, init2)
allom.parameters <- c ("a","b","mu.a","mu.b","sigma.y","sigma.a","sigma.b")
```

Run Jags

```
allom.4 <- jags(data = allom.data,
                inits = inits,
                parameters.to.save = allom.parameters,
                n.iter = 2500,
                model.file = here::here("slides","code","varintvarslope.bug"),
                n.chains = 2,
                n.burn = 1000)
#> Compiling model graph
#>    Resolving undeclared variables
#>    Allocating nodes
#> Graph information:
#>    Observed stochastic nodes: 488
#>    Unobserved stochastic nodes: 71
#>    Total graph size: 2521
#>
#> Initializing model
```

Display results

```
round(allom.4$BUGSoutput$summary[c(1:2, 32:33, 34:35, 65:66, 68:72), -c(4,6)],2)
#>            mean    sd   2.5%    50% 97.5% Rhat n.eff
#> a[1]       7.65  1.33   5.04   7.63 10.21 1.00  1600
#> a[2]      24.13  6.22  11.06  24.39 35.58 1.07    26
#> a[32]      8.39  1.88   4.81   8.35 12.10 1.00  3000
#> a[33]     13.22  3.67   6.18  13.26 20.09 1.00  1300
#> b[1]       1.87  2.89  -4.16   1.87  7.59 1.00  2200
#> b[2]     -11.04 10.82 -33.95 -10.52  9.03 1.09    23
#> b[32]      5.30  4.31  -3.06   5.22 13.88 1.00  3000
#> b[33]     13.27  7.06  -0.02  13.36 26.42 1.00  1400
#> mu.a      16.67  2.00  12.70  16.64 20.68 1.00  2900
#> mu.b       5.14  2.38   0.39   5.08  9.93 1.00  3000
#> sigma.a   10.78  1.49   8.29  10.65 14.04 1.00  3000
#> sigma.b   11.92  2.16   8.20  11.74 16.68 1.01  2400
#> sigma.y    2.66  0.09   2.49   2.66  2.83 1.00  3000
```

Compare with Frequentist approach

```
freq_lmm2 <- lmer (y ~ x + (1 + x | species), allom.data, REML = FALSE)
freq_lmm2
#> Linear mixed model fit by maximum likelihood  ['lmerMod']
#> Formula: y ~ x + (1 + x | species)
#>    Data: allom.data
#>       AIC       BIC    logLik -2*log(L)  df.resid
#>  2609.941  2635.083 -1298.971  2597.941       482
#> Random effects:
#>  Groups   Name        Std.Dev. Corr
#>  species  (Intercept) 10.409
#>           x           11.325   0.22
#>  Residual              2.652
#> Number of obs: 488, groups:  species, 33
#> Fixed Effects:
#> (Intercept)            x
#>      16.866        5.244
```

Compare with Frequentist approach - with no correlation

```
freq_lmm_wocorr <- lmer(y ~ x + (1 | species) +
                                    (0 + x | species), allom.data, REML = FALSE)
freq_lmm_wocorr
#> Linear mixed model fit by maximum likelihood  ['lmerMod']
#> Formula: y ~ x + (1 | species) + (0 + x | species)
#>    Data: allom.data
#>       AIC       BIC    logLik -2*log(L)  df.resid
#>  2609.086  2630.037 -1299.543  2599.086       483
#> Random effects:
#>  Groups    Name        Std.Dev.
#>  species   (Intercept) 10.203
#>  species.1 x           10.632
#>  Residual               2.661
#> Number of obs: 488, groups:  species, 33
#> Fixed Effects:
#> (Intercept)            x
#>      16.688        4.929
```

Shrinkage results from pooling of information

- Varying effect estimates shrink towards mean ($\bar{\alpha}$).

Shrinkage results from pooling of information

- Varying effect estimates shrink towards mean ($\bar{\alpha}$).
- Avoids underfitting as in complete pooling model (null variance) or overfitting as in no pooling model (infinite variance).

Shrinkage results from pooling of information

- Varying effect estimates shrink towards mean ($\bar{\alpha}$).
- Avoids underfitting as in complete pooling model (null variance) or overfitting as in no pooling model (infinite variance).
- Varying effects: adaptive regularization through cluster variance estimation.

Shrinkage results from pooling of information

- Varying effect estimates shrink towards mean ($\bar{\alpha}$).
- Avoids underfitting as in complete pooling model (null variance) or overfitting as in no pooling model (infinite variance).
- Varying effects: adaptive regularization through cluster variance estimation.
- Further from mean, more shrinkage.

Shrinkage results from pooling of information

- Varying effect estimates shrink towards mean ($\bar{\alpha}$).
- Avoids underfitting as in complete pooling model (null variance) or overfitting as in no pooling model (infinite variance).
- Varying effects: adaptive regularization through cluster variance estimation.
- Further from mean, more shrinkage.
- Fewer data in cluster, more shrinkage.

Multilevel models are awesome!

Multilevel models in a nutshell

- **Shrinkage via pooling is desirable**. The no-pooling model overstates variation among clusters and makes the individual clusters look more different than they are (overfitting). The complete-pooling model simply ignores the variation among clusters (underfitting).

Multilevel models in a nutshell

- **Shrinkage via pooling is desirable**. The no-pooling model overstates variation among clusters and makes the individual clusters look more different than they are (overfitting). The complete-pooling model simply ignores the variation among clusters (underfitting).
- We can **generalize to a wider population**. Is there an allometry relationship between number of seeds and biomass?

Multilevel models in a nutshell

- **Shrinkage via pooling is desirable**. The no-pooling model overstates variation among clusters and makes the individual clusters look more different than they are (overfitting). The complete-pooling model simply ignores the variation among clusters (underfitting).
- We can **generalize to a wider population**. Is there an allometry relationship between number of seeds and biomass?
- We may consider **varying slopes**. We'd need to deal with correlations between intercept and slope random effects. Open a whole new world with spatial (or time) autocorrelation, phylogenetic regressions, quantitative genetics, network models.

Multilevel models in a nutshell

- **Shrinkage via pooling is desirable**. The no-pooling model overstates variation among clusters and makes the individual clusters look more different than they are (overfitting). The complete-pooling model simply ignores the variation among clusters (underfitting).
- We can **generalize to a wider population**. Is there an allometry relationship between number of seeds and biomass?
- We may consider **varying slopes**. We'd need to deal with correlations between intercept and slope random effects. Open a whole new world with spatial (or time) autocorrelation, phylogenetic regressions, quantitative genetics, network models.
- We may **include predictors at the cluster level**. Imagine we know something about functional traits, and wish to determine whether some species-to-species variation in the allometry relationship is explained by these traits.

Your turn: Practicals 8 and 9